

Indian Institute of Technology Dharwad
CS201L: Artificial Intelligence Laboratory
Lab 10 – Regression with Multi-Input Variables

1 Problem Statement

You are given the dataset `AirQuality.csv`, which contains various air pollution indicators. Your task is to predict the **CO concentration** (mg/m^3) using regression models.

2 Dataset Details

Input Variables:

- PT08.S1(CO): Tin oxide sensor response (nominally CO targeted)
- NMHC: Non-Methanic HydroCarbons concentration ($\mu g/m^3$)
- C6H6(GT): Benzene concentration ($\mu g/m^3$)
- PT08.S2(NMHC): Titania sensor response (nominally NMHC targeted)
- NOx: NOx concentration (ppb)
- PT08.S3(NOx): Tungsten oxide sensor response (nominally NOx targeted)
- NO2(GT): NO₂ concentration ($\mu g/m^3$)
- PT08.S4(NO2): Tungsten oxide sensor response (nominally NO₂ targeted)
- PT08.S5(O3): Indium oxide sensor response (nominally O₃ targeted)
- T: Temperature (°C)
- RH: Relative Humidity
- AH: Absolute Humidity

Output Variable:

- CO concentration (mg/m^3)

3 Tasks

3.1 Data Splitting

Split the dataset into the following subsets:

- 60% Training Data
- 20% Validation Data
- 20% Test Data

3.2 Correlation Analysis

1. Compute the correlation between each input feature and the target variable **CO concentration** (y).
2. Compare the correlation coefficients and identify the feature that has the **highest absolute correlation** with the target variable.
3. Let this feature be denoted as x_{best} . This feature will be used as the independent variable for the regression tasks.

3.3 Ridge Regression (Polynomial Degree = 4)

1. Build a **Ridge Regression model** to predict **CO concentration** (y) using the selected feature x_{best} .
2. Transform the selected feature into polynomial features of degree 4.
3. Use Ridge Regression with regularization parameter:

$$\alpha = 1$$

4. Train the model on the training dataset and generate predictions for training, validation, and test datasets.
5. Compute the **RMSE** for all datasets:
 - Training RMSE
 - Validation RMSE
 - Test RMSE

3.4 Neural Network Regression (Top-2 Features)

1. Identify the **top-2 input features** based on highest absolute correlation with the target variable.
2. Use these two features as independent variables to predict **CO concentration** (y).
3. Build a neural network using PyTorch with the following architecture:
 - Input layer: 2 neurons
 - Hidden layer: 1 layer with **tanh** activation
 - Hidden neurons: {16, 32, 64}
 - Output layer: 1 neuron (linear output)
 - Loss function: Mean Squared Error (MSE)
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Learning rate: 0.001
 - Epochs: 500
4. Train models for each hidden layer configuration and compute:
 - Training RMSE
 - Validation RMSE
5. Plot **Training Loss vs Epochs** and **Validation Loss vs Epochs** for all configurations.
6. Select the model with the **lowest validation RMSE**.
7. Evaluate it on the test dataset and compute **Test RMSE**.

Sample Code Snippets for Regression Tasks

The following code blocks demonstrate how to implement the required correlation analysis, simple linear regression, and polynomial curve fitting using Python and scikit-learn.

1. **Correlation Analysis and Feature Selection:** Compute correlations with the target variable, identify the best feature, and prepare the splits.

```
1 import pandas as pd
2
3 # 1. Compute correlations with the target variable 'CO'
4 correlations = df.corr()['CO'].drop('CO')
5
6 # 2. Find the feature with the highest absolute correlation (x_best)
7 x_best = correlations.abs().idxmax()
8 print(f"Feature with highest absolute correlation: {x_best}")
9
10 # 3. Define independent variables using x_best
11
12 X_train = train_df[[x_best]]
13 y_train = train_df['CO']
14
15 X_val = val_df[[x_best]]
16 y_val = val_df['CO']
17
18 X_test = test_df[[x_best]]
19 y_test = test_df['CO']
```

2. **Ridge Regression (Degree = 4):**

```
1 from sklearn.linear_model import Ridge
2 from sklearn.preprocessing import PolynomialFeatures
3 from sklearn.metrics import mean_squared_error
4 import numpy as np
5
6 # Polynomial transformation
7 poly = PolynomialFeatures(degree=4)
8 X_train_poly = poly.fit_transform(X_train)
9 X_val_poly = poly.transform(X_val)
10 X_test_poly = poly.transform(X_test)
11
12 # Ridge Regression model
13 ridge = Ridge(alpha=1)
14 ridge.fit(X_train_poly, y_train)
15
16 # Predictions
17 y_train_pred = ridge.predict(X_train_poly)
18 y_val_pred = ridge.predict(X_val_poly)
19 y_test_pred = ridge.predict(X_test_poly)
20
21 # RMSE calculation
22 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
23 rmse_val = np.sqrt(mean_squared_error(y_val, y_val_pred))
24 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
25
26 print(f"Train RMSE: {rmse_train:.4f} | Val RMSE: {rmse_val:.4f} | Test RMSE: {rmse_test:.4f}")
```

3. Neural Network Regression (PyTorch):

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class Net(nn.Module):
6     def __init__(self, hidden_size):
7         super(Net, self).__init__()
8         self.fc1 = nn.Linear(2, hidden_size)
9         self.tanh = nn.Tanh()
10        self.fc2 = nn.Linear(hidden_size, 1)
11
12    def forward(self, x):
13        x = self.tanh(self.fc1(x))
14        x = self.fc2(x)
15        return x
16
17 hidden_sizes = [16, 32, 64]
18
19 for h in hidden_sizes:
20     model = Net(h)
21     criterion = nn.MSELoss()
22     optimizer = optim.SGD(model.parameters(), lr=0.001)
23
24     for epoch in range(500):
25         optimizer.zero_grad()
26         outputs = model(X_train_tensor)
27         loss = criterion(outputs, y_train_tensor)
28         loss.backward()
29         optimizer.step()
```

4. 3D Best-Fit Surface (Linear Regression with Top 2 Features):

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4
5 # Train Linear Regression model
6 model = LinearRegression()
7 model.fit(X_train_top2, y_train)
8
9 # Feature names
10 f1, f2 = X_train_top2.columns
11
12 # Create grid over feature space
13 x1 = np.linspace(X_train_top2[f1].min(), X_train_top2[f1].max(), 100)
14 x2 = np.linspace(X_train_top2[f2].min(), X_train_top2[f2].max(), 100)
15
16 X1_grid, X2_grid = np.meshgrid(x1, x2)
17
18 # Predict surface
19 grid_points = np.c_[X1_grid.ravel(), X2_grid.ravel()]
20 Z_pred = model.predict(grid_points)
21 Z_pred = Z_pred.reshape(X1_grid.shape)
22
23 # 3D Plot
24 fig = plt.figure(figsize=(10,7))
25 ax = fig.add_subplot(111, projection='3d')
```

```

26
27 # Plot surface
28 surface = ax.plot_surface(X1_grid, X2_grid, Z_pred,
29                           cmap='viridis', alpha=0.7)
30
31 # Plot training data
32 ax.scatter(X_train_top2[f1],
33            X_train_top2[f2],
34            y_train,
35            color='red', s=20, label='Training Data')
36
37 # Labels and title
38 ax.set_xlabel(f1)
39 ax.set_ylabel(f2)
40 ax.set_zlabel("Heating Load")
41 ax.set_title("3D Best-Fit Surface (Linear Regression)")
42
43 # Colorbar and view
44 fig.colorbar(surface, ax=ax, shrink=0.5, aspect=10)
45 ax.view_init(elev=30, azim=45)
46
47 ax.legend()
48 plt.show()
49
50 # Print model equation
51 w1, w2 = model.coef_
52 b = model.intercept_
53
54 print(f"z = {w1:.4f}*{f1} + {w2:.4f}*{f2} + {b:.4f}")

```